

SQL, NoSQL & How Data Is Stored

recap of what we covered, plus your questions for tomorrow



Dinesh Rawat

Instructor · Shorter Loop × Snapiid

This is a recap of what we covered today: the words behind databases, and the difference between the two big families of them. We did not finish the whole plan, so the last part of today is four questions I left with you to research and present tomorrow. Read this tonight so the ideas are fresh, and so the presenters have a running start.

1 What is a database, and what is a DBMS

A **database** is an organised collection of data. That is all the word means: data, kept in order so you can find it. But the data does not manage itself. Something has to store it, protect it, and answer questions about it. That something is the DBMS.

DBMS — Database Management System

The software that stores and manages a database for you. It handles saving data, reading it back, keeping it safe, and letting many people use it at once. MySQL is a DBMS. So are PostgreSQL, Oracle, and MongoDB. When you "install MySQL", you are installing a DBMS.

So the **database** is the data. The **DBMS** is the program that looks after it. People often say "database" for both, but now you know the difference.

2 What is a language, and what is a query

You talk to a DBMS in a **language**. Just as you write JavaScript to talk to a browser, you write database commands in a database language. The most common one is SQL.

Query

A single question or command you send to the database. "Give me all users from Haldwani" is a query. "Add this new user" is a query. Every time your app reads or writes data, it sends a query. The word comes from "to ask": a query asks the database to do something.

So a **language** is how you speak to the database, and a **query** is one thing you say in that language.

3 SQL vs NoSQL: the two families

There are two big families of databases. The difference is how they organise data.

SQL databases

Store data in `tables` with fixed columns, like a spreadsheet. Every row has the same shape. You must decide the columns in advance. MySQL is a SQL database. SQL is also the name of the language these use.

NoSQL databases

Store data more freely, often as flexible documents, where different records can have different fields. You do not have to fix the shape in advance. MongoDB is the best known NoSQL database.

	SQL (e.g. MySQL)	NoSQL (e.g. MongoDB)
shape	tables, fixed columns	flexible documents
every record	same columns	can differ
plan the shape first?	yes	not required
good for	data with a clear, steady structure	data that changes shape a lot

Neither is better. They are tools for different jobs. Our resume project uses MySQL, a SQL database, because a user and a resume have a clear, steady shape.

4 RDBMS: where the "relational" comes in

RDBMS — Relational Database Management System

A DBMS for SQL databases, where tables can be *related* to each other. A user table and a resume table, linked so each resume knows its user. The "R" is that link. MySQL is an RDBMS.

You already met the link itself: the foreign key, the `userId` column that ties a resume to its user. That relationship between tables is exactly what the "relational" in RDBMS means. So MySQL is a DBMS (it manages data), a SQL database (it uses tables and the SQL language), and an RDBMS (its tables relate to each other). All three words describe the same tool from different angles.

5 Structured vs unstructured data

This is the idea underneath the SQL and NoSQL split.

Structured data

Data with a fixed, predictable shape. A users table where every row has a name, an email, and a password, and nothing else. It fits neatly into rows and columns. SQL databases are built for structured data.

Unstructured data

Data with no fixed shape: a photo, a video, a long free-text note, a chat message. It does not fit tidy columns. This is where flexible NoSQL stores often help.

Structured	Unstructured
a user's name and email	a profile photo
a resume's title	a voice recording
an order's total price	a free-text support chat

Tie it together. Structured data has a shape you can predict, so it fits a SQL table. Unstructured data does not, so it often lives in a NoSQL store or as a file. Our resume fields (name, email, title) are structured, which is why MySQL fits the project.

6 CHAR vs VARCHAR vs the STRING type

When you store text in MySQL you pick how. Three names came up, and they are easy to mix up.

CHAR

Fixed length. `CHAR(10)` always takes room for 10 characters, even if you store just 3, padding the rest with spaces. Fast when every value is genuinely the same length, like a country code "IN" or a fixed-length id.

VARCHAR

Variable length. `VARCHAR(255)` can hold up to 255 characters and only uses the room it needs. Store "Himanshu" and it takes eight characters worth, not 255. This is what you use for almost all normal text: names, emails, titles.

	CHAR(n)	VARCHAR(n)
length	always n, padded	up to n, only what is used
best for	fixed-size codes	everyday text of varying length
example	country code, pin length	name, email, title

Where **STRING** fits. `STRING` is not a MySQL word, it is a **Sequelize** word. When you write `DataTypes.STRING` in a model, Sequelize turns it into `VARCHAR(255)` in MySQL. So `STRING` is the JavaScript name, `VARCHAR` is the real MySQL column it becomes. Same thing, two layers. Rule of thumb: `VARCHAR` for text that varies in length, `CHAR` only when every value is exactly the same size.

how each one stores the name "Deepesh"

Storing "Deepesh" (7 letters) in a size-10 text column

CHAR(10) always reserves 10 cells

D	e	e	p	e	s	h	.	.	.
---	---	---	---	---	---	---	---	---	---

= 10 used

3 blank spaces added (padding)

VARCHAR(10) stores only what you use

D	e	e	p	e	s	h	nothing stored here		
---	---	---	---	---	---	---	---------------------	--	--

= 7 used

Same name, less space.

`CHAR` wastes 3 cells on padding every time. `VARCHAR` keeps only the 7 it needs.

Deepesh has 7 letters. In a size-10 column, `CHAR` fills all 10 cells and pads the 3 extra with blank spaces, every time. `VARCHAR` keeps only the 7 the name uses. Store a million names and `CHAR` is storing a million rows of padding you never needed.

7 More MySQL types: `ENUM`, `BOOLEAN`, `TINYINT`

Text is not the only kind of column. A few more types came up, each for a specific job.

`ENUM`

A column that can only hold one value from a fixed list you decide in advance. A `status` column might be `ENUM('active', 'pending', 'deleted')`. Try to save anything else and MySQL refuses. It keeps a column honest when there are only a few valid options.

`BOOLEAN`

A true or false column. `isVerified`, `termsAccepted`. It answers a yes-or-no question about the row.

TINYINT

A very small whole number. Here is the connection: MySQL has no separate true/false type underneath, so `BOOLEAN` is actually stored as `TINYINT(1)`, a tiny number that is 1 for true and 0 for false. When you ask for `BOOLEAN`, MySQL gives you a `TINYINT(1)`. Two names, one storage.

Type	Holds	Use for
<code>ENUM</code>	one value from a fixed list	status, role, provider
<code>BOOLEAN</code>	true or false	isVerified, isActive
<code>TINYINT</code>	a small number (0-255)	the storage behind <code>BOOLEAN</code> , small counts

Why this matters for design. Picking the tightest type that fits your data is good design. A status is not free text, so make it an `ENUM` and the database guards it. A yes-or-no is a `BOOLEAN`, not a string "yes". The type is your first line of defence for clean data.

8 Why learn MySQL commands, when Sequelize writes SQL for us?

Fair question. If Sequelize writes the SQL, why learn to write it yourself? Because knowing the commands underneath makes you far more capable, and here is exactly why.

- **You can debug.** When a Sequelize query behaves strangely, you turn on logging, read the SQL it generated, and run it yourself in Workbench to see what is really happening. Without SQL, that error is a black box.
- **You can check your data directly.** "Did that row actually save?" You open Workbench and run `SELECT`. No app needed. You go straight to the source of truth.
- **You understand what the ORM is doing.** Sequelize is a convenience over SQL, not a replacement for understanding it. When you know the SQL, the ORM stops being magic and becomes a tool you control.
- **Not every job uses Sequelize.** SQL is the same everywhere: MySQL, PostgreSQL, in Python, in Java, in a data analyst role. The ORM changes between projects. SQL is the skill that transfers to all of them.

The honest summary. The ORM writes SQL so you do not have to, every day. But when something breaks, when you need to check the truth, or when you move to a project or job without an ORM, SQL is what saves you. Learn the tool and the layer beneath it. That is the difference between someone who can use a library and someone who understands their craft.

9 Your questions for tomorrow

We did not finish today, so these four are yours to research and present tomorrow. Each of you takes one. The point is not just the story. It is what it teaches about how data and websites really work, and how to keep

them safe. Come ready to explain it simply to the rest of the batch.

Himanshu — How a class 12 CBSE student was able to get into the CBSE portal and change marks. Focus on: what weakness let it happen, and what should have stopped it. This is a lesson in why security matters, not a how-to.

Deepesh — How someone could create 10,000+ records in a short time. Focus on: what a script or bot is, and why a site needs limits and checks so this cannot happen.

Divya Bora — What information is kept in the UI, the HTML, CSS and JavaScript that runs in the browser. Focus on: what is visible to anyone who opens "view source", and therefore what must never be trusted or hidden there.

Gaurav and Ayush — Who are ethical hackers. Focus on: what they do, why companies pay them, and how it differs from illegal hacking.

The thread running through all four. Notice they connect. Weak security lets marks be changed. No limits let a bot flood records. Whatever the browser holds is open to everyone. And ethical hackers are the people who find these holes before the bad actors do. Tomorrow, as a group, you are really answering one question: how do we build software that does not break in these ways.

Everything here ran live in class. If a part does not click, message me the file name and the line, and read it again tonight while it is fresh.